DEPI · SYSTEM ANALYSIS & DESIGN · DELIVERABLE 6 OF 6

Additional Deliverables

Section 4.6 — API documentation, testing and validation plan, deployment strategy

MediCare — Clinic Management & Appointment System

ASP.NET Core MVC · N-Tier · Microsoft Azure

Team: Mohamed Munir (Team Lead) · Malak Hany · Ashraf Ibrahim · Leqaa Ibrahim · Natalie Bola

November 2026

16. API Documentation

MediCare is an ASP.NET Core MVC app: most routes return Razor views; routes marked **JSON** return data for FullCalendar.js and Chart.js. Route names are the proposed design. *(Optional)* marks optional features.

1. General

Item	Value
Base URL (local)	`https://localhost:<port>`
Base URL (deployed)	`https://<app-name>.azurewebsites.net`
Layers	MediCare.Web → MediCare.Services → MediCare.Data
Controllers	Account, Doctors, Appointments, MedicalRecords, Prescriptions, Admin
Areas	Patient, Doctor, Admin
Authentication	ASP.NET Core Identity (cookie), `[Authorize(Roles = ...)]`
CSRF	Anti-forgery token on every POST
Date format	ISO 8601, e.g. `2026-11-20T10:30:00`
Input	ViewModels and DTOs only; entities never bound directly

Enumerations

- AppointmentStatus: Pending, Confirmed, Rejected, Cancelled, Completed (*Completed per RTM FR-09; not yet in the State Diagram*)
- AppointmentType: Consultation, Follow-up, Emergency (*Architecture doc lists Examination once and Emergency once; pick one set*)
- Role: Patient, Doctor, Admin

Status codes: 200 success · 302 redirect after form submit · 400 validation error · 401 not logged in · 403 role not allowed / not your record · 404 not found · 409 slot already booked.

2. Account – `AccountController`

Method	Route	Role	Description	FR
GET/POST	`/Account/Login`	Public	Login; redirects to the role's dashboard	FR-02, FR-03
GET/POST	`/Account/Register`	Public	Patient registration	FR-01
GET/POST	`/Account/RegisterDoctor`	Public	Doctor registration; inactive until admin approves	FR-01
POST	`/Account/Logout`	Any	Logout	FR-02

Invalid login returns the login page with an error; invalid registration returns field-level errors.

3. Doctors – `DoctorsController`

Method	Route	Role	Description	FR
GET	`/Doctors?specialization=&location=&date=`	Patient	Search/filter doctors	FR-05
GET	`/Doctors/Details/{id}`	Patient	Doctor profile, specializations, fee	FR-05
GET	`/api/doctors/{id}/availability?start=&end=` **JSON**	Patient	Available slots for FullCalendar (working hours − vacation days − booked slots)	FR-06

```
[
  { "start": "2026-11-20T10:00:00", "end": "2026-11-20T10:30:00", "title": "Available" },
  { "start": "2026-11-20T10:30:00", "end": "2026-11-20T11:00:00", "title": "Available" }
]
```

4. Patient Area

4.1 Appointments – `Patient/AppointmentsController`

Method	Route	Description	FR
POST	`/Patient/Appointments/Book`	Submit booking request; status starts Pending	FR-07, FR-08
GET	`/Patient/Appointments`	Manage appointments (upcoming and past)	FR-09
POST	`/Patient/Appointments/{id}/Reschedule`	Move to a new available slot	FR-11
POST	`/Patient/Appointments/{id}/Cancel`	Cancel an eligible appointment; frees the slot	FR-11
POST	`/Patient/Waitlist/Join` *(Optional)*	Join waitlist when no slot	FR-18
POST	`/Patient/Waitlist/{id}/Respond` *(Optional)*	Accept/decline a slot offer before deadline	FR-18
POST	`/Patient/Waitlist/{id}/Leave` *(Optional)*	Leave the waitlist	FR-18

`POST /Patient/Appointments/Book` (AJAX from FullCalendar)

```
{ "doctorId": 7, "start": "2026-11-20T10:00:00", "end": "2026-11-20T10:30:00", "type":
"Consultation", "notes": "First visit" }
```

Success `200`: `{ "success": true, "appointmentId": 301, "status": "Pending" }` Conflict `409` (unique constraint on Doctor + DateTime, checked in `IAppointmentService.BookAsync`): `{ "success": false, "error": "This slot is no longer available.", "waitlistAvailable": true }` `400` if the slot is outside working hours or on a vacation day.

4.2 Medical information

Method	Route	Description	FR
GET	`/Patient/MedicalRecords`	View own medical history	FR-12
GET	`/Patient/MedicalRecords/{id}/Documents/{docId}`	Download a medical document (Blob Storage)	FR-12
GET	`/Patient/Prescriptions`	Prescription history	FR-14
GET	`/Patient/Prescriptions/{id}`	One prescription (print-friendly)	FR-14

4.3 Profile

Method	Route	Description	FR
GET/POST	`/Patient/Profile`	View/update personal info, allergies	FR-04
GET/POST	`/Patient/Profile/NotificationPreferences`	Choose notification channels	FR-17

5. Doctor Area

5.1 Appointments and schedule

Method	Route	Description	FR
GET	`/Doctor/Dashboard`	Daily/weekly appointments	FR-10
GET	`/api/doctor/appointments?start=&end=` **JSON**	Calendar events	FR-10

Method	Route	Description	FR
POST	`/Doctor/Appointments/{id}/Accept`	Pending → Confirmed	FR-10
POST	`/Doctor/Appointments/{id}/Reject`	Pending → Rejected	FR-10
POST	`/Doctor/Appointments/{id}/Complete`	Confirmed → Completed	FR-09
GET/POST	`/Doctor/Schedule`	Working hours (DayOfWeek, AvailableFrom, AvailableTo)	FR-06
POST	`/Doctor/Schedule/Vacations`	Add/remove vacation days	FR-06
GET/POST	`/Doctor/Profile`	Professional profile	FR-04

Allowed status transitions (invalid → `400`)

From	To	By
(new)	Pending	Patient books
Pending	Confirmed / Rejected	Doctor
Pending / Confirmed	Cancelled	Patient
Confirmed	Completed	Doctor

5.2 Medical records – `Doctor/MedicalRecordsController`

Method	Route	Description	FR
GET	`/Doctor/Patients/{patientId}/MedicalRecords`	History, previous prescriptions, allergies	FR-12
GET/POST	`/Doctor/MedicalRecords/Create?appointmentId=`	Create record (RecordDate, Diagnosis, Notes)	FR-13
POST	`/Doctor/MedicalRecords/{id}/Edit`	Update record; audited	FR-13, NFR-06
POST	`/Doctor/MedicalRecords/{id}/Documents`	Upload test results/X-rays (multipart/form-data)	FR-13

5.3 Prescriptions – `Doctor/PrescriptionsController`

Method	Route	Description	FR
GET/POST	`/Doctor/Prescriptions/Create?recordId=`	Create prescription (medication, dosage, frequency)	FR-14
POST	`/api/prescriptions/safety-check` **JSON** *(Optional)*	Check interactions, contraindications, allergies for a draft	FR-19
POST	`/Doctor/Prescriptions/{id}/Approve`	Approve and save; updates record, notifies patient	FR-14
GET	`/Doctor/Prescriptions/{id}/Print`	Print-friendly view	FR-14

Safety-check request/response:

```
        { "patientId": 12, "items": [ { "medication": "Medication A", "dosage": "500 mg", "frequency": "Twice daily" } ] }
```

```
        { "conflict": true, "details": [ "Interaction with a current medication" ], "alternatives": [ "Medication B" ] }
```

If `conflict` is true the doctor edits the draft and the check re-runs; Approve is allowed only with no conflict. The source of interaction/allergy rules is still to be defined.

6. Admin Area

Method	Route	Description	FR
GET	`/Admin/Dashboard`	Overview	FR-15
GET	`/Admin/Users`	List users	FR-15
POST	`/Admin/Users/{id}/ToggleAccess`	Enable/disable account	FR-15
GET	`/Admin/Doctors`	List doctors	FR-15
POST	`/Admin/Doctors/{id}/Approve`	Approve doctor registration	FR-15
POST	`/Admin/Doctors/{id}/Edit`	Update doctor info	FR-15
GET/POST	`/Admin/Specializations`	List, add, edit specializations	FR-15
GET	`/Admin/Appointments`	View all appointments	FR-15
GET/POST	`/Admin/Waitlist/Rules` *(Optional)*	Configure response deadline and queue rules	FR-18

Reports and analysis

Method	Route	Description	FR
GET	`/Admin/Reports`	Reports page with Chart.js	FR-16
GET	`/api/reports/appointments?year=` **JSON**	Monthly statistics	FR-16
GET	`/api/reports/popular-doctors` **JSON**	Most popular doctors	FR-16
GET	`/api/reports/popular-specializations` **JSON**	Most popular specializations	FR-16
GET	`/api/reports/revenue?from=&to=` **JSON**	Revenue summary	FR-16
GET	`/Admin/Reports/Export?type=pdf|excel&report=`	Export (QuestPDF, ClosedXML)	FR-16

Example `GET /api/reports/appointments?year=2026`: `{ "labels": ["Jan","Feb","Mar"], "values": [120, 98, 143] }`

7. Real-time notifications – `NotificationHub` (SignalR)

Hub URL `/hubs/notifications` · logged-in users only; each user joins a group named by their user id · production via Azure SignalR Service · client method `ReceiveNotification(payload)`.

Event type	Sent to	Trigger
NewAppointment	Doctor	Patient books
AppointmentConfirmed	Patient	Doctor accepts
AppointmentRejected	Patient	Doctor rejects
AppointmentRescheduled / AppointmentCancelled	Doctor and Patient	Change by the patient
Reminder	Patient	24 h before (Hosted Service or Hangfire)
PrescriptionReady	Patient	Prescription approved
WaitlistOffer *(Optional)*	Patient	A slot opens; response timer starts

Payload example: `{ "type": "AppointmentConfirmed", "appointmentId": 301, "start": "2026-11-20T10:00:00" }`
Delivery channel chosen by `NotificationFactory` from patient preferences: SignalR (instant), SendGrid (email), Twilio (SMS). Keys live in Azure Key Vault.

8. Access matrix

Area	Public	Patient	Doctor	Admin
Login, register	✔	✔	✔	✔

Area	Public	Patient	Doctor	Admin
Search doctors, view availability		✔		
Book, reschedule, cancel, waitlist		✔		
Accept, reject, schedule, vacations			✔	
Write medical records and prescriptions			✔	
Read own records and prescriptions		✔		
Read records of own patients			✔	
Users, doctors, specializations, reports				✔

A patient reads only their own data; a doctor only their own patients' records. Enforce in the Services layer (NFR-01), not only with `[Authorize]`.

17. Testing & Validation Plan

> Proves MediCare is built correctly (verification) and solves the right problem for patients, doctors and admins (validation). Follows the four project weeks.

Scope

In scope: multi-role auth and registration; doctor search/filter (specialization, location, availability); booking, cancellation, rescheduling, double-booking prevention; doctor dashboard (schedule, working hours, vacation days, confirm/reject); SignalR notifications, Twilio SMS, SendGrid email, 24-hour reminders; medical records, file uploads, prescriptions with print view, patient profile; admin dashboard, Chart.js analytics, PDF/Excel export; responsive UI, dark/light mode, accessibility; Azure deployment and CI/CD. **Out of scope:** testing Azure/Twilio/SendGrid themselves (mocked in automated tests, checked once with sandbox accounts) and any real patient data.

Objectives

1. Every functional requirement has at least one passing test case.
2. No double booking, even when two users book the same slot at the same moment.
3. A user can only see data their role allows.
4. Notifications arrive: SignalR within seconds, SMS/email at the scheduled time.
5. Deployed Azure system behaves like the local build.
6. A patient, a doctor and an admin accept the system in UAT.

Test strategy

Level	What it checks	Tools	Runs
Unit	Business rules in MediCare.Services (conflict detection, slot generation, fee/status rules), Factory classes	xUnit, Moq, FluentAssertions	Every commit
Data / repository	Repositories and Unit of Work: CRUD, filters, failed save rolls back related changes	xUnit with SQLite in-memory or SQL Server test container	Every commit
Integration	Controllers, services, EF Core, Identity over HTTP	ASP.NET Core WebApplicationFactory, Postman/Newman	Every commit
System / end-to-end	Complete journeys in a real browser incl. FullCalendar and SignalR	Playwright or Selenium	Nightly and on staging
Acceptance (UAT)	Real users confirm fit	Scripted scenarios, feedback form	End of Week 4 on staging
Regression	Old features still work	Re-run automated suites	Every pipeline run

Test types: functional, security/authorization, performance/load, concurrency, usability/accessibility, browser/device compatibility, deployment smoke.

Mocking external services: Twilio, SendGrid and the clock sit behind interfaces (`ISmsSender`, `IEmailSender`, `IDateTimeProvider`); tests use fakes, so no real SMS is sent and "24 hours before" can be tested instantly. **Design-pattern checks:** Repository/Unit of Work (second save fails → nothing saved); Factory (each appointment/notification type returns the right object); DI (every controller resolves its services at startup).

Test environments and data

Environment	Where	Used for
Local / CI	Dev machine and pipeline agent, test DB, fake SMS/email	Unit, data, integration
Staging	App Service staging slot with own Azure SQL, SignalR Service, Twilio/SendGrid sandbox	E2E, performance, security scan, UAT
Production	Live Azure App Service	Smoke test only after each release

Test data (seed scripts, all fake): 3 admins, 10 doctors across ≥4 specializations with different schedules/vacations, 30 patients (some with allergies/long histories), appointments in every status.
Browsers/devices: latest Chrome, Edge, Firefox, Safari + one Android and one iPhone size.

Functional test cases

ID	Feature	Steps and input	Expected result	Level	Status
TC-01	Register patient	Valid name, email, password	Account created with role Patient, redirected to login	Integration	Not run
TC-02	Register patient	Existing email or weak password	Validation error, no account	Integration	Not run
TC-03	Register doctor	Specialization, fee, profile image	Account created as Doctor, pending admin approval	Integration	Not run
TC-04	Login by role	Log in as Patient, Doctor, Admin	Each lands on own dashboard	System	Not run
TC-05	Authorization	Patient opens admin/doctor URL	Access denied (403 or redirect)	Integration	Not run
TC-06	Doctor search	Filter by specialization, location, availability	Only matching doctors; empty result shows message	Integration	Not run
TC-07	Unit of Work	Two related changes, second fails	Both rolled back	Data	Not run
TC-08	Booking	Pick free slot in FullCalendar and confirm	Saved as Pending, slot no longer free	System	Not run
TC-09	Conflict detection	Book an already-taken slot	Rejected with clear message	Unit, Integration	Not run
TC-10	Concurrent booking	Two patients book same slot at once	Exactly one succeeds	Integration	Not run
TC-11	Past and off-hours	Past time, vacation day, outside hours	Slot not offered or rejected	Unit	Not run
TC-12	Cancel	Patient cancels upcoming appointment	Cancelled, slot freed, doctor notified	System	Not run
TC-13	Reschedule	Move to another free slot	Old slot freed, new held, no overlap	System	Not run
TC-14	Doctor confirm/reject	Confirm one, reject another	Statuses update, patient notified	System	Not run
TC-15	Working hours	Doctor sets hours and a vacation day	Calendar shows only valid slots	System	Not run
TC-16	SignalR to doctor	Patient books while doctor online	Doctor sees notification without refresh	System	Not run
TC-17	SignalR to patient	Doctor confirms/rejects	Patient sees notification live	System	Not run

ID	Feature	Steps and input	Expected result	Level	Status
TC-18	SignalR offline user	Notification while offline	Shown after next login (stored)	Integration	Not run
TC-19	Medical record	Doctor adds diagnosis, symptoms, notes	Saved, linked to patient and appointment	Integration	Not run
TC-20	File upload	Upload PDF and X-ray; try disallowed/oversized	Valid stored and downloadable; invalid refused	Integration	Not run
TC-21	Record privacy	Patient A requests Patient B's record	Access denied	Integration	Not run
TC-22	Prescription	Two medications with dosage/frequency	Saved, in patient's history	Integration	Not run
TC-23	Print prescription	Open print view	Clean one-page layout, no nav	System (manual)	Not run
TC-24	SMS reminder	Appointment 24 h ahead, fake clock advanced	Exactly one SMS request to the sender interface	Unit	Not run
TC-25	Email notification	Confirm an appointment	One email request, correct recipient/content	Unit	Not run
TC-26	Notification preferences	Patient turns SMS off	No SMS, email still sent	Unit	Not run
TC-27	Patient profile	Update personal info, allergies, history	Saved and shown on reload	System	Not run
TC-28	Admin approves doctor	Approve pending doctor	Doctor can log in and appears in listing	System	Not run
TC-29	Admin management	Block a patient; edit specialization; view all appointments	Blocked patient cannot log in; changes show	System	Not run
TC-30	Charts	Open analytics with seeded data	Totals equal DB counts/sums	Integration	Not run
TC-31	Export	Export report to PDF and Excel	Files open and match screen	System	Not run
TC-32	Dark/light mode	Toggle, reload	Mode persists, pages readable	System	Not run

Non-functional tests (targets are proposed — agree final numbers with the supervisor)

ID	Area	Test	Target	Tool
NF-01	Performance	Doctor search and booking pages under 50 concurrent users	< 2 s, no errors	k6, JMeter or Azure Load Testing
NF-02	Load/concurrency	100 simultaneous attempts on the same slot	Exactly one booking, no server errors	k6 or integration test
NF-03	Security: access	Change IDs in URLs to read others' data	Always denied	Manual, Postman
NF-04	Security: web attacks	Scan SQLi, XSS, CSRF; anti-forgery tokens	No medium+ findings	OWASP ZAP
NF-05	Security: accounts	Password rules, lockout, HTTPS-only cookies	Enforced as configured	Manual, integration test
NF-06	Security: secrets	Search repo and logs for keys	None found	Pipeline secret scanner
NF-07	Data protection	Files not public by direct URL; TLS	Unauthenticated download refused	Manual

ID	Area	Test	Target	Tool
NF-08	Usability	3–5 users book unaided	Completed unaided, difficulties logged	Observation
NF-09	Accessibility	Keyboard-only, screen reader, contrast	No critical issues; WCAG 2.1 AA goal	axe DevTools, Lighthouse, manual
NF-10	Compatibility	Core journeys on 4 browsers and 2 phone sizes	Same behaviour	Playwright, BrowserStack, real devices
NF-11	Reliability	Restart App Service during use	Recovers, SignalR reconnects, no data lost	Manual on staging
NF-12	Deployment	Migrations on staging, swap slot	Smoke test passes; rollback by swapping back works	Pipeline smoke test

Validation – user acceptance testing (end of Week 4, staging)

ID	Role	Scenario	Accepted when
UAT-01	Patient	Find a cardiologist near me, book Tuesday morning, reschedule	Done without help in reasonable time
UAT-02	Patient	View past visits, read a prescription, download a lab result	All found without help
UAT-03	Patient	Receive and understand reminder SMS and email	Both arrive, details correct
UAT-04	Doctor	Set next week's hours and a vacation day, handle three requests	Calendar correct, patients notified live
UAT-05	Doctor	Open a patient, add record and prescription, print	Prescription complete and printable
UAT-06	Admin	Approve a doctor, review last month's stats, export a report	Numbers right, export opens

Feedback: did the task match how you work? what was confusing? what is missing? ease of use 1–5. **Sign-off:** all UAT scenarios completed, average ease-of-use ≥ 4/5, no open critical/high defects; product owner or supervisor signs with a date.

Test schedule by project week

Week	Features delivered	Testing
1	N-Tier setup, schema, repositories, authentication, doctor listing	Service unit tests, repository/UoW tests, auth tests (TC-01–07)
2	Booking, calendar, doctor dashboard, SignalR	Booking, conflict, concurrency, notification tests (TC-08–18, NF-02)
3	Records, prescriptions, Twilio, SendGrid, patient profile	Records, upload, privacy, notifications with fakes (TC-19–27, NF-03, NF-07)
4	Admin, analytics, UI polish, Azure deployment	Admin/report tests, regression, performance, security scan, accessibility, staging smoke, UAT (TC-28–32, NF-01, NF-04–12, UAT-01–06)

Entry and exit criteria

Entry: build compiles, unit tests pass in pipeline, test DB seeded, build deployed to target environment.
Exit: 100% of high-priority cases run and pass; ≥95% of all pass; no open critical/high defects (medium/low listed with a decision); NF targets met or gaps accepted in writing; UAT sign-off recorded; production smoke test passes.

Roles

Role	Responsibility
Developers	Unit/integration tests for own code, fix defects
Test lead	Owns the plan, reviews results, decides pass/fail with team

Role	Responsibility
Testers	Manual and E2E tests, report defects, retest
Product owner / supervisor	Joins UAT and signs off
DevOps owner	Pipeline, environments, staging data

Risks and mitigation

Risk	Mitigation
Double booking under simultaneous requests	DB-level uniqueness/concurrency check; TC-10 and NF-02 on every build
Time-based features hard to test	Clock behind an interface, fake time
Twilio/SendGrid limits, cost, outages	Fakes in automated tests, sandbox accounts for manual checks
Exposure of medical data	Role checks on every query, NF-03 to NF-07, fake data only
Staging differs from production	Same infrastructure setup, migrations applied by pipeline
Little time in Week 4	Automated tests from Week 1, run on every commit

Defect management

Log in GitHub Issues, Azure Boards or Jira with: title, steps, expected/actual, environment/build, screenshot, severity, owner. Life cycle: New → Assigned → Fixed → Retested → Closed (or Reopened).

Severity	Meaning	Example	Fix target
Critical	System unusable, data loss/exposure	Patient reads another's record	Before any further testing
High	Core feature broken, no workaround	Booking fails for every user	Before release
Medium	Works with a workaround	Reschedule works only after refresh	Before release if time allows
Low	Cosmetic	Misaligned button in dark mode	Backlog

MediCare – Deployment Strategy

1. Introduction

The deployment strategy defines how the MediCare Clinic Management & Appointment System will be prepared, configured, deployed, secured, and maintained in a production environment.

MediCare is an ASP.NET Core MVC web application built using a three-tier N-Tier architecture. The system will be deployed on Microsoft Azure to provide patients, doctors, and administrators with secure access to its features through a web browser.

The deployment approach aims to ensure:

- Reliable access to the system.
- Secure storage and handling of medical information.
- Proper communication between application components.
- Automated application updates through CI/CD.
- Database backup and recovery.
- Easy maintenance and future scalability.

2. Deployment Architecture

MediCare will use a cloud-based deployment architecture hosted on Microsoft Azure.

The architecture consists of the following components:

2.1 Client Layer

Users access the system through a web browser using a computer, tablet, or mobile device.

The system supports three main user roles:

- **Patient:** Searches for doctors, books appointments, and accesses medical records and prescriptions.
- **Doctor:** Manages appointments, working hours, medical records, and prescriptions.

- **Admin:** Manages users, doctors, specializations, appointments, and reports.

2.2 Application Hosting – Azure App Service

The ASP.NET Core MVC application will be hosted using Azure App Service.

It will contain the three logical application layers:

- **MediCare.Web:** Handles HTTP requests, user interfaces, controllers, and presentation logic.
- **MediCare.Services:** Handles business logic, validation, and application workflows.
- **MediCare.Data:** Handles database operations using Entity Framework Core, repositories, and Unit of Work.

These layers communicate through Dependency Injection.

The application will be published as a deployable ASP.NET Core application and hosted by Azure App Service.

2.3 Database Hosting – Azure SQL Database

Azure SQL Database will be used to store the system's persistent data, including:

- User accounts and roles.
- Doctors and patients.
- Specializations.
- Appointments and working hours.
- Medical records.
- Prescriptions and related information.

Entity Framework Core will manage database access and schema migrations.

2.4 Real-Time Communication – Azure SignalR Service

Azure SignalR Service will support real-time communication between the application and connected users.

It will be used for notifications such as:

- New appointment requests.
- Appointment confirmations and rejections.
- Appointment-related updates and reminders.

2.5 External Notification Services

The system will integrate with:

- **Twilio:** For sending SMS notifications.
- **SendGrid:** For sending email notifications.

These services will be accessed through securely configured credentials.

2.6 File Storage

Medical documents and uploaded files must be stored in a controlled storage location.

Recommended deployment option: Azure Blob Storage can be used to store uploaded medical documents and test results. The database should store file metadata and storage references rather than the file contents themselves.

Access to medical documents must be authorized through the application.

3. Hosting Environment

The proposed production environment is:

Component	Hosting / Technology
Web Application	Azure App Service
Database	Azure SQL Database
Real-Time Communication	Azure SignalR Service
Medical File Storage	Azure Blob Storage (recommended)

Component	Hosting / Technology
SMS Notifications	Twilio
Email Notifications	SendGrid
Source Code	GitHub
CI/CD	GitHub Actions or Azure DevOps
Application Framework	ASP.NET Core MVC
ORM	Entity Framework Core

Azure App Service will host the application, while Azure SQL Database will manage persistent data independently.

The external notification providers will be accessed through their respective APIs.

4. Deployment Process

The deployment process will follow these stages:

Step 1: Prepare the Application

- Complete the implementation of the Web, Services, and Data layers.
- Verify Dependency Injection and application configuration.
- Ensure that Entity Framework Core migrations are ready.
- Verify that authentication and role-based authorization work correctly.
- Test integrations with SignalR, Twilio, and SendGrid.

Step 2: Prepare the Azure Environment

- Create an Azure App Service instance.
- Create an Azure SQL Database instance.
- Configure Azure SignalR Service.
- Create Azure Blob Storage if it is selected for medical document storage.
- Configure the required application settings and connection strings.

Step 3: Configure the Database

- Create the production database.
- Configure the database connection.
- Apply the required Entity Framework Core migrations.
- Insert necessary initial data, such as roles, the initial administrator, and starter specializations.

Database migrations should be reviewed and applied through a controlled deployment process.

Step 4: Build and Publish the Application

- Restore NuGet dependencies.
- Build the ASP.NET Core application.
- Run automated tests.
- Publish the application in Release configuration.
- Prepare the published output for deployment.

Step 5: Deploy the Application

- Deploy the published application to Azure App Service.
- Configure the production environment.
- Enable HTTPS.
- Verify the application can connect to Azure SQL Database and the external services.

Step 6: Verify the Deployment

Perform smoke tests to confirm that:

- The application loads successfully.

- Users can log in and access their authorized dashboards.
- Doctors can manage appointments.
- Patients can book appointments.
- Medical records and prescriptions can be accessed according to permissions.
- Real-time notifications work.
- Email and SMS integrations work when configured.
- Uploaded documents can be accessed by authorized users.

Step 7: Release to Production

After successful verification, the production deployment can be considered complete.

5. CI/CD Strategy

MediCare will use Continuous Integration and Continuous Deployment (CI/CD) through GitHub Actions or Azure DevOps.

The proposed pipeline is:

1. A developer pushes code changes to the GitHub repository.
2. The CI/CD pipeline is triggered.
3. Dependencies are restored.
4. The application is built.
5. Automated tests are executed.
6. If all checks pass, the application is published.
7. The published application is deployed to Azure App Service.
8. Post-deployment health checks are performed.

Database migrations should be handled as a controlled step rather than being applied blindly on every deployment.

A staging environment is recommended to test changes before releasing them to production.

6. Configuration Management

The application requires environment-specific configuration.

The following settings must be configured securely:

- Database connection string.
- ASP.NET Core environment.
- Azure SignalR connection settings.
- Twilio account credentials.
- SendGrid API key.
- File storage connection settings, if Azure Blob Storage is used.

Sensitive values must not be committed to GitHub or stored directly in source code.

Recommended practices:

- Use Azure App Service application settings for environment-specific configuration.
- Use Azure Key Vault for managing sensitive secrets.
- Use Managed Identity where supported to reduce the need for stored credentials.
- Keep separate configuration values for development, staging, and production.

7. Security Strategy

Since MediCare handles sensitive healthcare information, security is a major deployment requirement.

The following measures are recommended:

7.1 Authentication and Authorization

- Use ASP.NET Identity for authentication.

- Apply role-based authorization for Patient, Doctor, and Admin.
- Restrict access to medical records and prescriptions.
- Ensure users can only access information they are authorized to view.

7.2 Secure Communication

- Enforce HTTPS for all application traffic.
- Use secure connections between the application and database.
- Avoid exposing database credentials or API keys.

7.3 Data Protection

- Protect sensitive configuration values.
- Apply appropriate database access permissions.
- Validate uploaded files and restrict permitted file types and sizes.
- Ensure medical documents are not publicly accessible.

7.4 Application Security

- Validate user input on the server side.
- Handle exceptions without exposing sensitive technical details.
- Keep application dependencies updated.
- Apply security updates regularly.

8. Backup and Recovery Strategy

A backup and recovery plan is necessary to protect patient information and maintain system availability.

The proposed strategy includes:

- Enable Azure SQL Database automated backups.
- Define a suitable backup retention period based on operational and project requirements.
- Regularly verify that backups are available.
- Test database restoration in a separate environment.
- Back up uploaded medical documents separately if external file storage is used.
- Document the recovery procedure.

In case of database failure or accidental data loss, the database can be restored using the available Azure backup capabilities.

The team should periodically test the recovery process instead of assuming that backups alone guarantee successful recovery.

9. Monitoring and Maintenance

After deployment, the system requires continuous monitoring and maintenance.

Recommended activities include:

- Monitor application availability.
- Review application errors and logs.
- Monitor database performance and resource usage.
- Check notification service failures.
- Monitor file storage usage.
- Update application dependencies.
- Apply security patches.
- Review user access and permissions.
- Test backup restoration periodically.

Azure Monitor and Application Insights are recommended tools for application monitoring, diagnostics, and performance analysis.

10. Rollback Strategy

If a deployment introduces critical errors, the team should be able to return to a previously working application version.

The rollback procedure includes:

1. Identify the deployment issue using logs and health checks.
2. Stop further releases until the issue is understood.
3. Redeploy the last stable application version.
4. Verify that the application is functioning correctly.
5. Restore the database only if necessary and after evaluating the impact of data changes.

Database schema changes should be designed carefully because rolling back application code does not automatically reverse database migrations.

Azure App Service deployment slots may be used if supported by the selected hosting plan.

11. Deployment Validation Checklist

Before considering the deployment successful, verify the following:

- Application is accessible through its HTTPS URL.
- Database connection is successful.
- EF Core migrations have been applied.
- Initial roles and administrator account are configured.
- Patient, Doctor, and Admin authentication works.
- Role-based authorization is enforced.
- Appointment booking and conflict detection work.
- Medical records and prescriptions are accessible to authorized users.
- SignalR notifications work.
- Twilio SMS integration works, if configured.
- SendGrid email integration works, if configured.
- Medical document uploads and access work.
- Application logs and monitoring are available.
- Database backup and recovery procedures are documented.

12. Conclusion

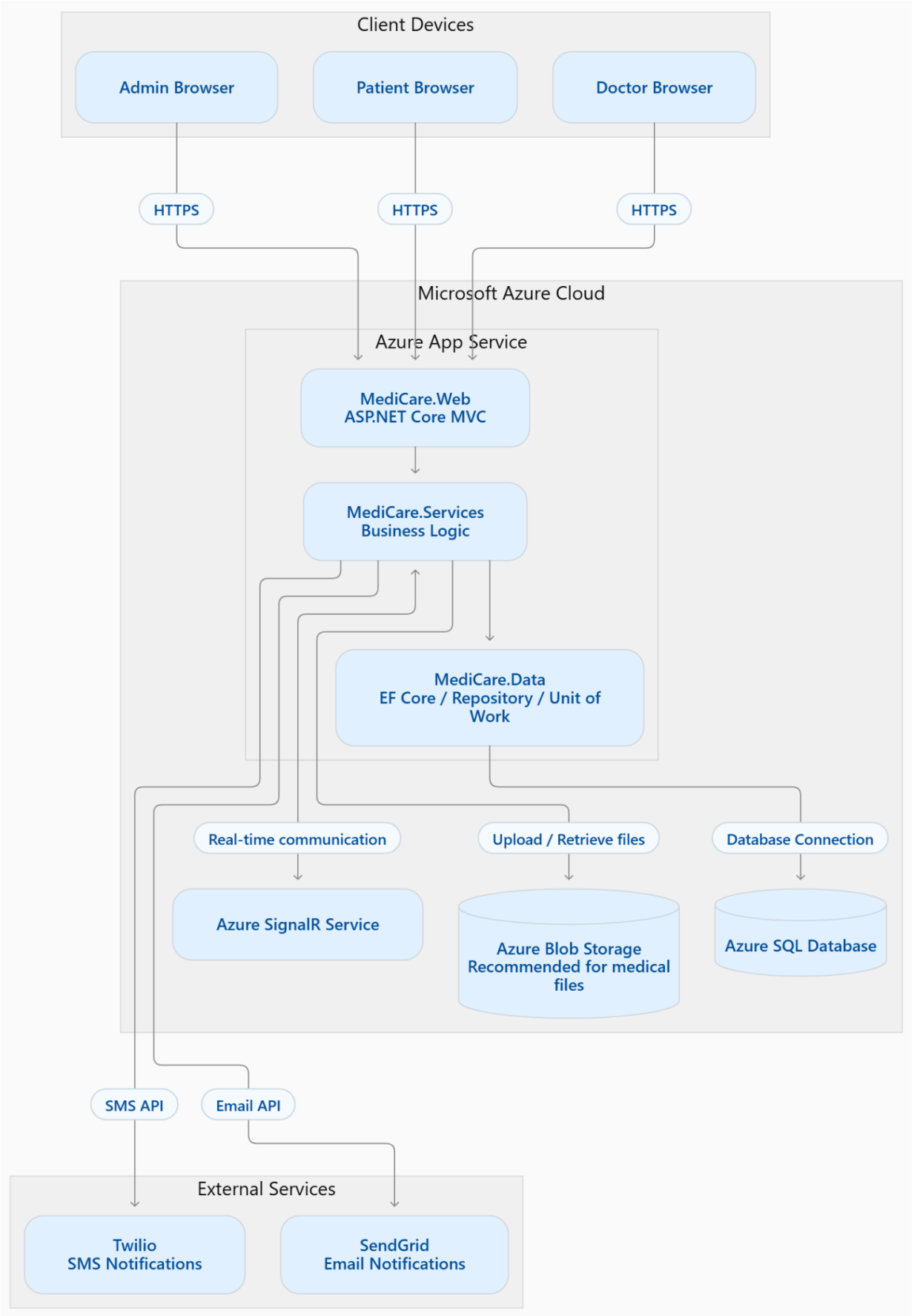
The proposed deployment strategy uses Microsoft Azure to host the MediCare application and its supporting services.

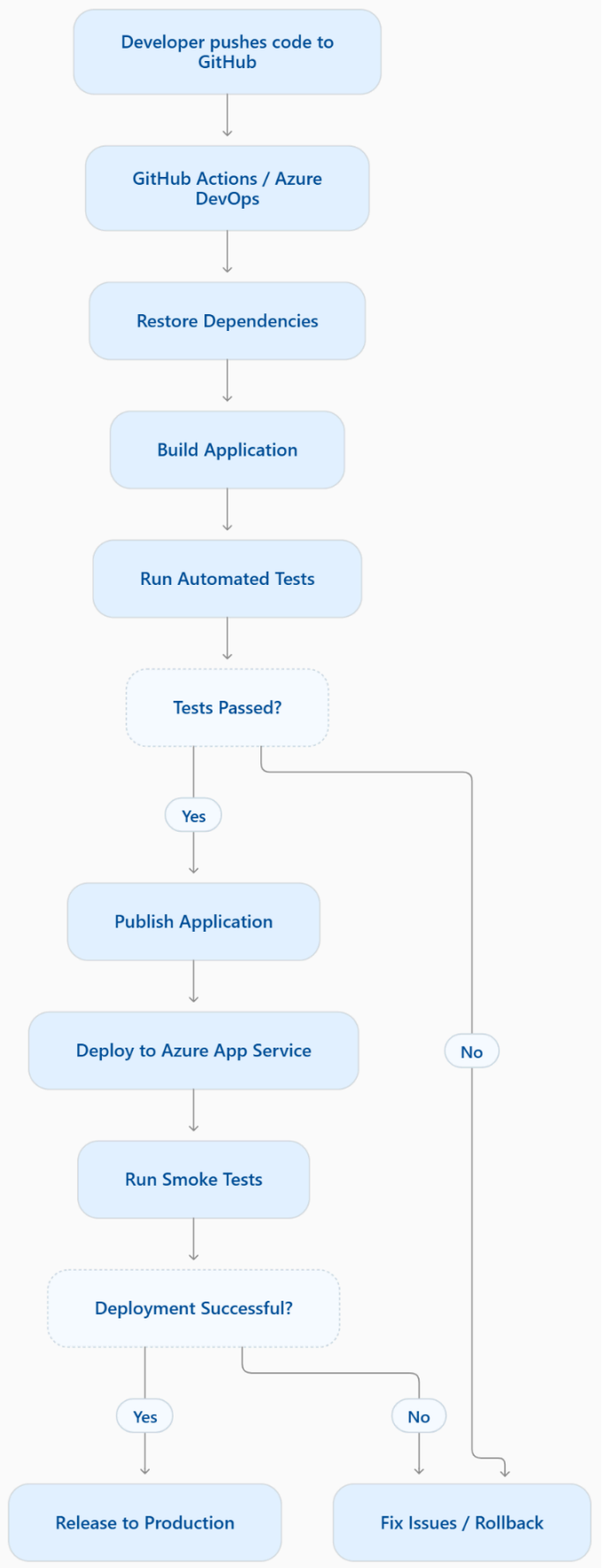
Azure App Service will host the ASP.NET Core MVC application, Azure SQL Database will store persistent data, and Azure SignalR Service will provide real-time communication. Twilio and SendGrid will support SMS and email notifications.

CI/CD through GitHub Actions or Azure DevOps will help automate the build, testing, and deployment process.

By combining secure configuration, role-based access control, backups, monitoring, and rollback procedures, the deployment strategy aims to provide a reliable and maintainable production environment for MediCare.

Deployment Architecture Diagram


Client Devices
Admin Browser
Patient Browser
Doctor Browser
HTTPS
HTTPS
HTTPS
Microsoft Azure Cloud
Azure App Service
MediCare.Web
ASP.NET Core MVC
MediCare.Services
Business Logic
MediCare.Data
EF Core / Repository / Unit of Work
Real-time communication
Upload / Retrieve files
Database Connection
Azure SignalR Service
Azure Blob Storage
Recommended for medical files
Azure SQL Database
SMS API
Email API
External Services
Twilio
SMS Notifications
SendGrid
Email Notifications



CI/CD Pipeline Diagram